

Criando Efeitos Visuais para Jogos

Utilizando Shader e XNA

Bruno Pereira Evangelista

www.brunoevangelista.com



Gamefest'07
MICROSOFT GAME TECHNOLOGY CONFERENCE BRASIL

Agenda

- Introdução
- Shaders
- Pipeline de Renderização
- Evolução dos Shaders
- Como Programar Shaders
- Programando Shaders com XNA
- Ferramentas
- Conclusões

Introdução

- Durante muitos anos as APIs gráficas, como DirectX e OpenGL, utilizaram um fluxo fixo de renderização
 - Não era possível modificar os algoritmos utilizados na renderização, como: transformação, iluminação, texturização
 - Só era permitido modificar um pequeno número de parâmetros fixos utilizados durante a renderização
- Resultado
 - Jogos com recursos gráficos muitos parecidos

Introdução

- Enquanto isso, a indústria do cinema possuía ferramentas que permitiam modificar vários estágios do processamento das cenas
- RenderMan
 - Especificação de uma linguagem de shaders, criada pela Pixar
 - Primeira versão publicada em 1988
 - Atualmente existem várias implementações, incluindo implementações open-source
- No entanto, a maioria dessas ferramentas era utilizada para renderização off-line

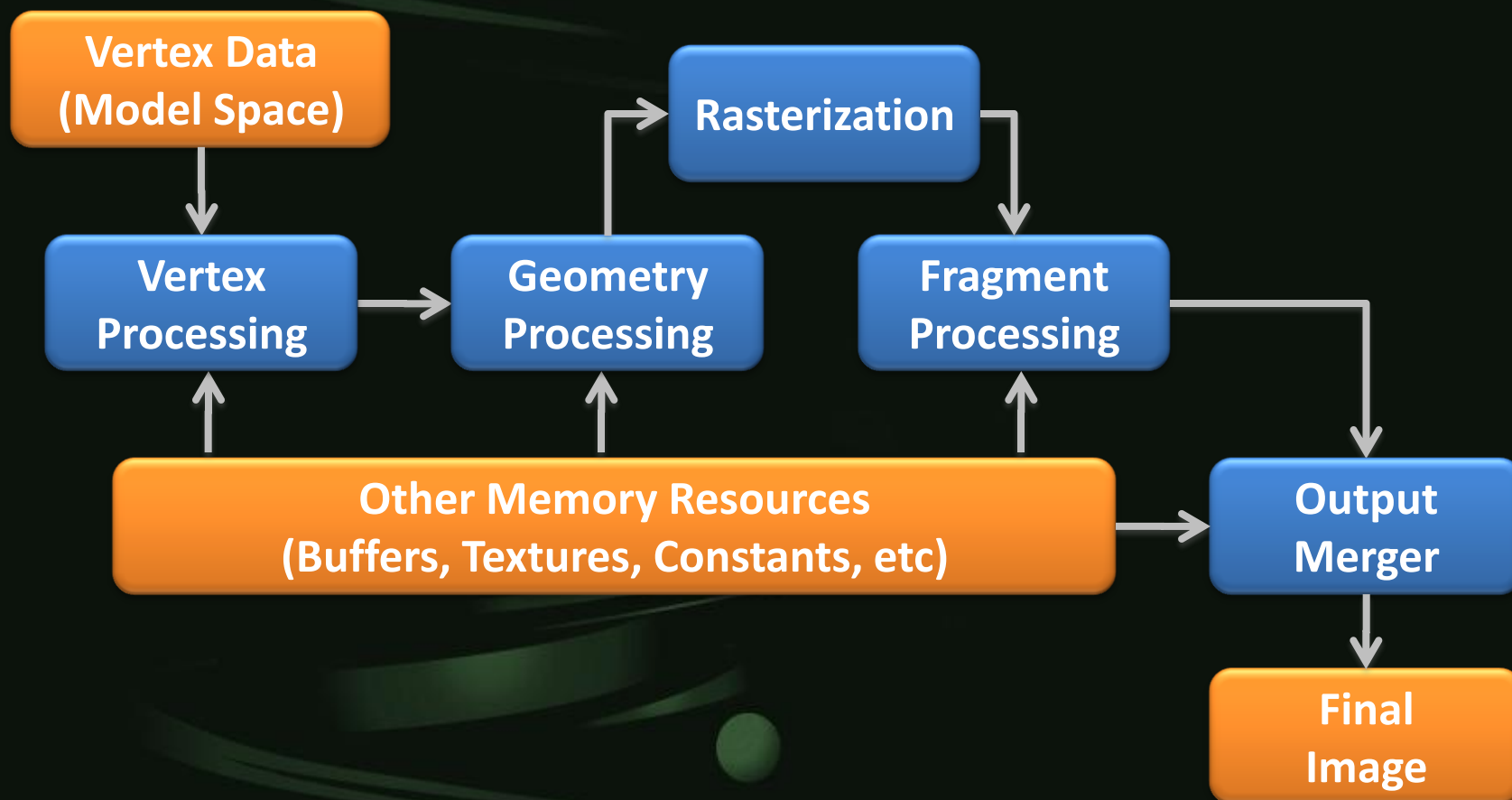
Shaders

- São pequenos programas que são executados no hardware gráfico programável (GPU)
- Permitem modificar alguns estágios do fluxo de renderização
- Novas possibilidades
 - ▣ Modelos de iluminação reais
 - ▣ Renderização de superfícies detalhadas
 - ▣ Efeitos de pós-processamento
- Além, da possibilidade de utilizar tudo isso em tempo real !!!

Shaders

- **Diferentes tipos de shaders são utilizados para programar diferentes estágios da renderização**
 - **Vertex Shader – Especifica como os vértices serão processados**
 - **Pixel Shader – Especifica como os pixels/fragmentos gerados serão processados**
 - **Geometry Shader* – Especifica como as primitivas serão processadas, além de permitir criar novas primitivas**
- **Veremos onde cada shaders é utilizado, no pipeline de renderização**

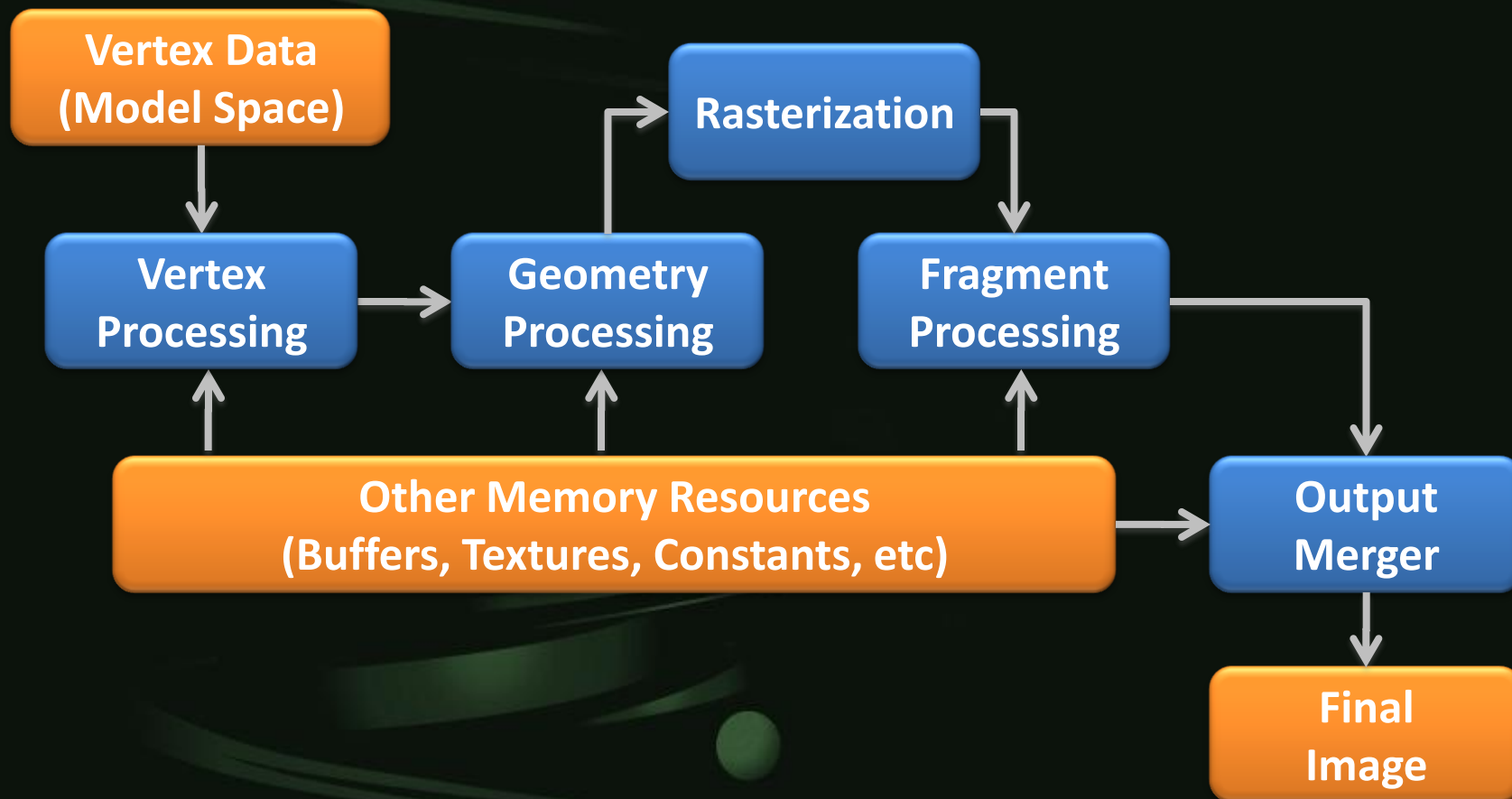
Pipeline de Renderização



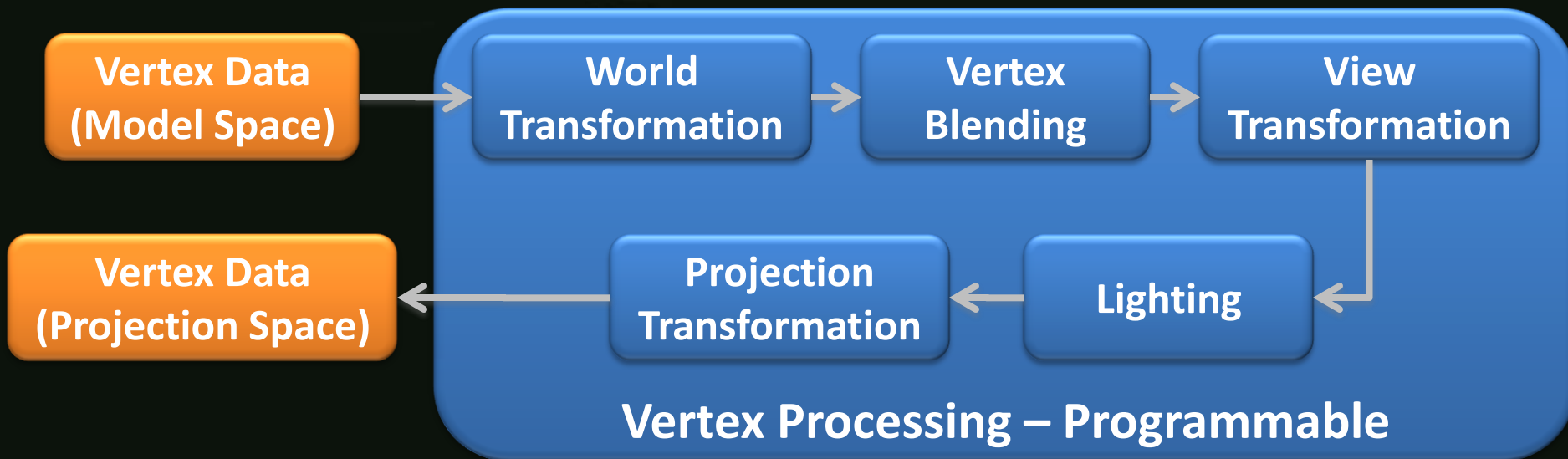
Pipeline de Renderização

- Responsável em transformar o modelo da cena em uma matriz de pixels que é exibida para o usuário
- Entrada
 - Informações sobre os vértices da malha do modelo: Posição, cor, normal, coordenada de textura, etc
- Saída
 - Imagem na forma de uma matriz de pixels

Pipeline de Renderização



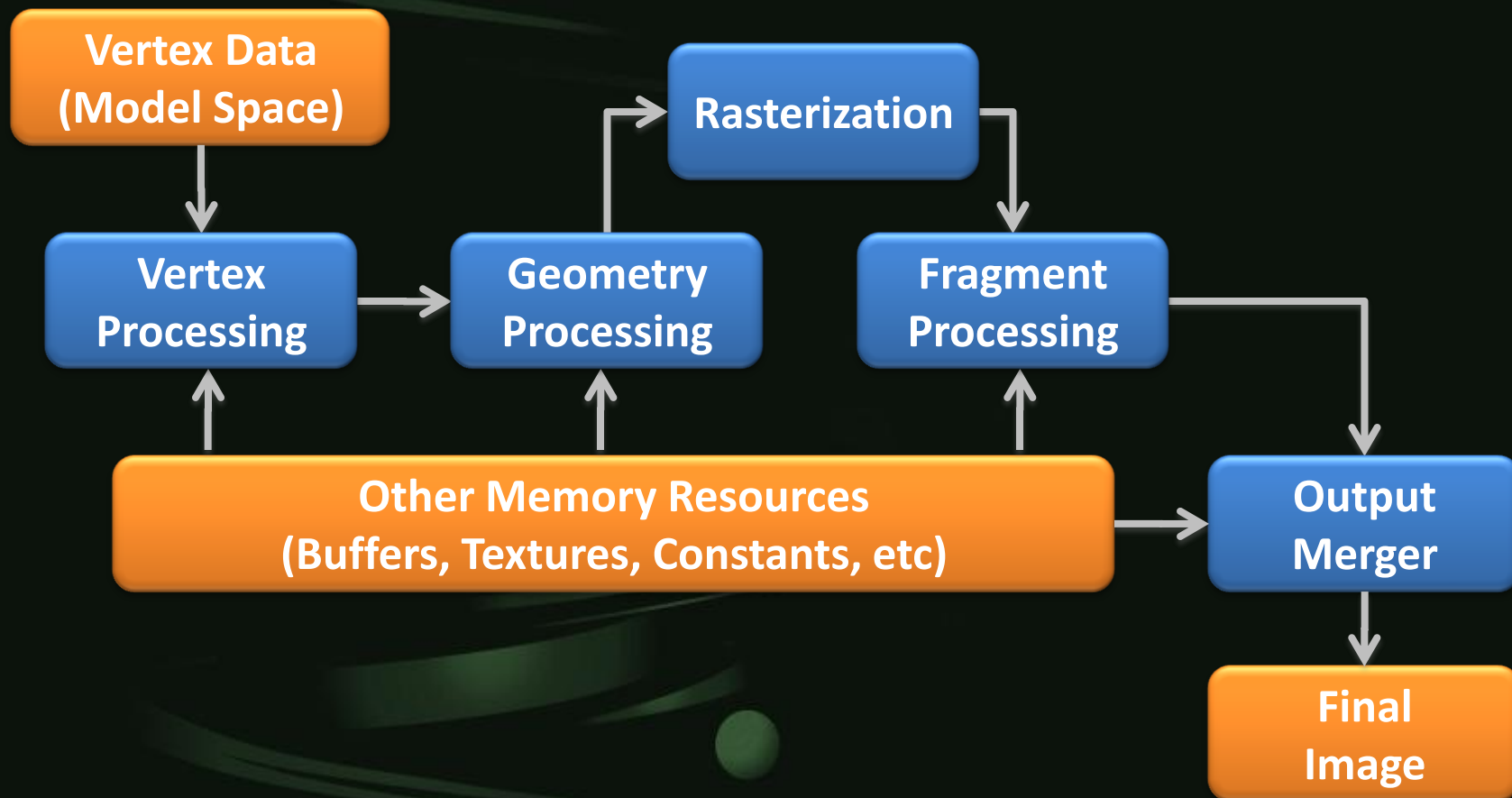
Processamento de Vértices



Processamento de Vértices

- Responsável em gerar a posição final de cada vértice na cena
 - Recebe os dados das malhas da cena enviados pela aplicação
- Possíveis usos
 - Deformação de malhas / Animação Esqueletal
 - Iluminação por vértice / Fog por vértice

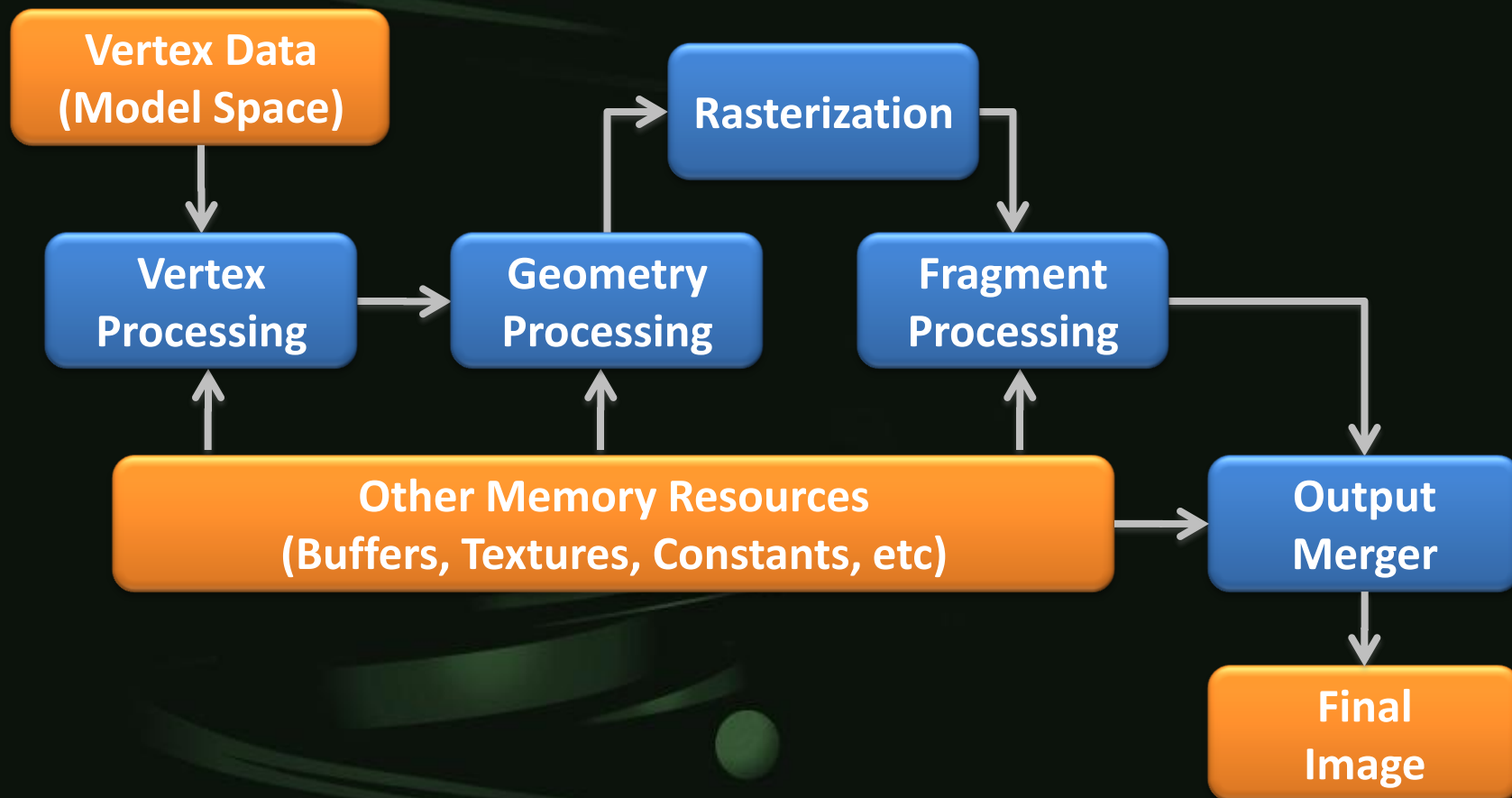
Pipeline de Renderização



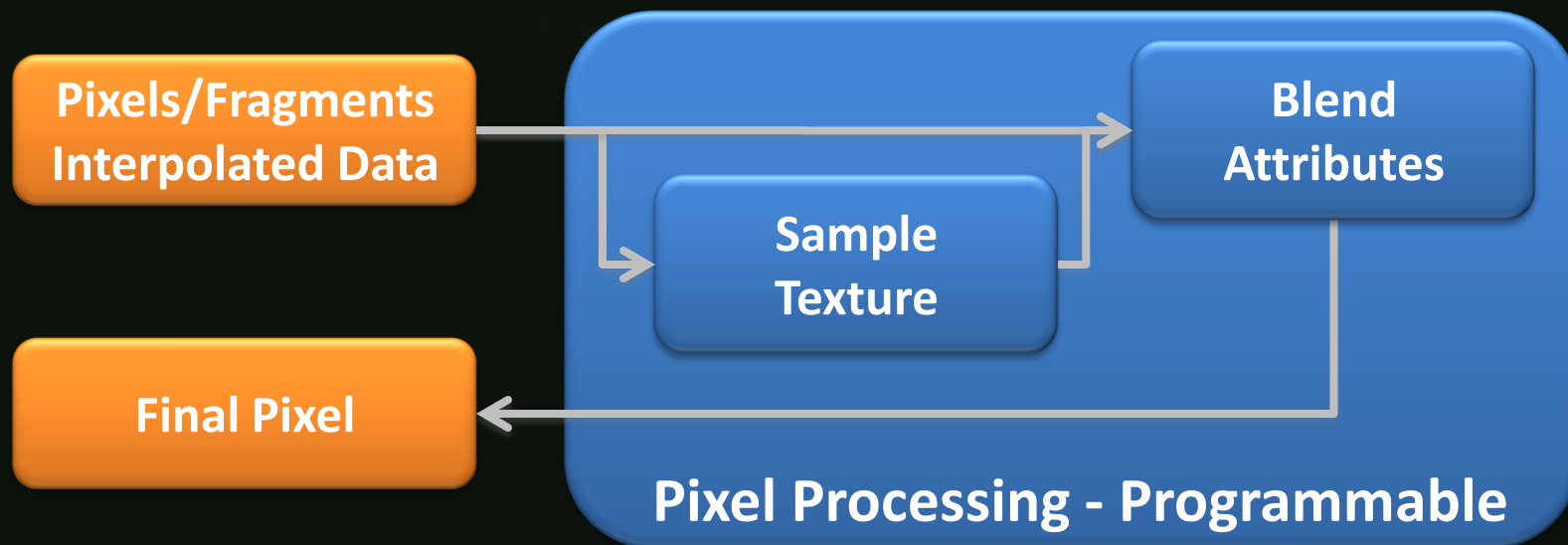
Rasterização

- Rasteriza as primitivas, transformando vértices em um conjunto de pixels/fragmentos
 - Primitivas fora do volume de visão são cortadas
 - Cada fragmento gerado possui as informações dos vértices interpolados

Pipeline de Renderização



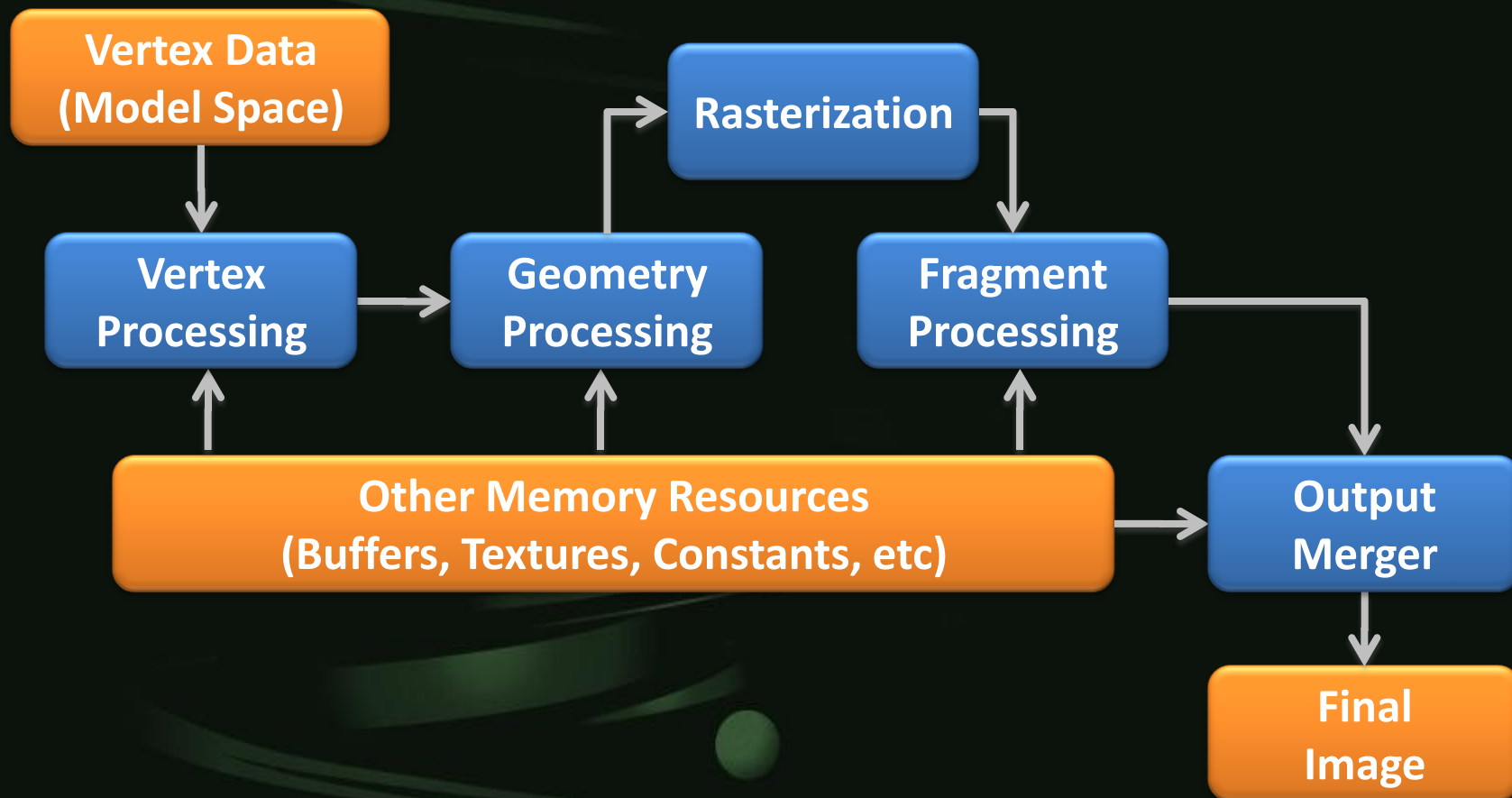
Processamento de Pixels



Processamento de Pixels

- Responsável em gerar a cor final de cada pixel
 - Recebe os dados de saída do processamento de vértices, interpolados
- Possíveis usos
 - Iluminação por pixel
 - Normal Mapping, Relief Mapping
 - PDI (Contraste, Brilho, Saturação)
 - Pós-processamento: Motion Blur, Bloom
- Os dados da saída podem ou não ser gravados no framebuffer

Pipeline de Renderização



A Evolução dos Shaders

Demos: Particles VS, Skinned VS, Material/Light PS

Videos: Azureus Temple, Boid Simulator, XNA FPS



Como Programar Shaders?

- Existem diferentes linguagens para se programar shaders
 - Linguagens de baixo nível, assembly
 - Linguagens de alto nível, similares ao C

Como Programar Shaders?

- **Linguagens de alto nível**
 - **HLSL (High Level Shading Language):** Desenvolvida pela Microsoft e o padrão utilizado no DirectX e XNA
 - **GLSL (OpenGL Shading Language):** Desenvolvida pela 3DLabs é o padrão utilizado no OpenGL
 - **Cg (C for Graphics):** Linguagem desenvolvida pela nVidia, muito similar ao HLSL. Pode ser utilizada no DirectX e OpenGL

Como Programar Shaders?

Assembly Language

vs_1_1

Version Instruction

```
// constants set by the application
// c0-c3 - View+Projection matrix
// c4.x - 1
// c4.y - 0
// c4.z - 0.5

// constants assembled into the shader
def c8, 0, 1, 2, 3
def c9, 0.0f, 0.25f, 0.5f, 0.75f
...
def c24, 1,1,1,1

dcl_position v0
dcl_texcoord0 v1

// output the texture coordinates
mov oT0, v1

// Transform position
dp4 oPos.x, v0, c0
dp4 oPos.y, v0, c1
dp4 oPos.z, v0, c2
dp4 oPos.w, v0, c3
```

```
// Transform position
dp4 oPos.x, v0, c0
dp4 oPos.y, v0, c1
dp4 oPos.z, v0, c2
dp4 oPos.w, v0, c3
```

Input Register Declarations

Instructions

Como Programar Shaders?

HLSL

```

struct vertexInput {
    float4 pos      : // Vertex in clip space and world space
    float3 normal   :
    float2 uv0      :
};

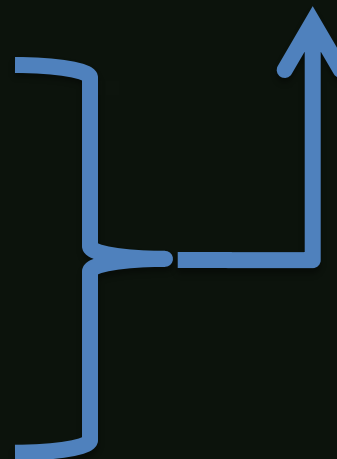
struct vertexOutput {
    float4 hpos      : // Normal, Eye and Light vector
    float2 uv0      :
    float3 eyeVec    :
    float3 lightVec  :
    float3 normalVec :
};

vertexOutput simple_vs
vertexOutput OUT;

// Vertex in clip space and world space
OUT.hpos = mul(IN.pos, matWVP);
float3 vertexPos = mul(IN.pos, matW);

// Normal, Eye and Light vector
OUT.normalVec = mul(IN.normal, matW);
OUT.eyeVec = matVI[3].xyz - vertexPos;
OUT.lightVec = lightPos - vertexPos;
OUT.uv0 = IN.uv0 * tile;
return OUT;
}

```



Efeitos

- **Efeitos são entidades que encapsulam vários shaders**
 - Permite definir várias técnicas diferentes para renderização
 - Cada técnica pode compartilhar processamentos de vértice, pixel e geometria
 - A técnica a ser utilizada pode ser definida com base no hardware gráfico do usuário
- **Os efeitos também permitem modificar os estados do pipeline de renderização**

Programando Shaders com XNA

- Diferente do DirectX e do OpenGL o XNA não possui um pipeline fixo de processamento
 - Na GPU do Xbox 360 não existe um pipeline fixo
 - Não é possível criar uma aplicação no XNA que não utilize shaders! xD
- Existem classes helpers no XNA que encapsulam o uso de shaders
 - BasicEffect
 - SpriteBatch

Programando Shaders com XNA

- Os recursos fornecidos pelas classes helpers do XNA são ainda mais limitados que o pipeline fixo
 - Mais cedo ou mais tarde você terá que criar seus shaders! =)

Programando Shaders com XNA

- Utilizar shaders no XNA é fácil !!!
 - O pipeline de conteúdo do XNA possui suporte a efeitos no formato HLSL
 - Os efeitos são compilados, gerando arquivos otimizados para o Windows ou Xbox 360
 - Esses arquivos podem ser facilmente carregados pelo gerenciador de conteúdo do XNA

```
Content.Load<Effect> (@ "Shaders\NormalMap.fx")
```

Ferramentas

- Existem várias ferramentas que auxiliam a criação de shaders
- FX Composer 2.0
 - Desenvolvido pela nVidia
 - Excelente ferramenta para criação de shaders, auxilia programadores e artistas
 - Suporta o formato COLLADA

Programando Shaders com XNA

- Criando um shader de iluminação por pixel no FX Composer
 - Na verdade já trouxe ele pronto! =)
- Carregar e utilizar o shader numa renderização no XNA

Outros Exemplos

- Detailed Surface Rendering
- Cartoon Rendering
- PDI (Contraste, Brilho, Saturação)

Conclusões

- **Desenvolvimento de shaders é um tópico que se tornou muito importante nos últimos**
 - **Possibilidade de crias vários efeitos para os jogos**
 - **Grande área de pesquisa e investimento em várias empresas**
 - **Área excitante**
 - **anos foi a programação e desenvolvimento de shaders**

Conclusões

- Em um futuro próximo, todas as APIs gráficas deverão abandonar o pipeline fixo
 - Isso já é realidade no XNA e no DirectX 10
- Estágios da renderização que ainda são fixos devem se tornar programáveis
 - Rasterização
 - Output Merger

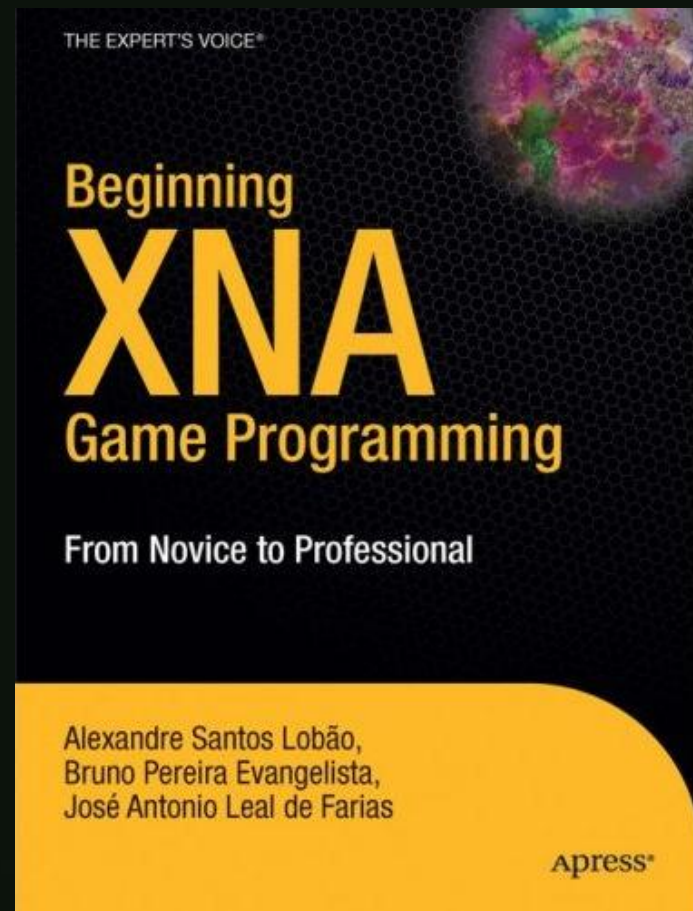
Perguntas?

Bruno P. Evangelista

bpevangelista@gmail.com

Home Page

www.brunoevangelista.com



"De fato, que aproveitará ao homem ganhar o mundo inteiro mas perder sua alma?" Mateus 16, 26